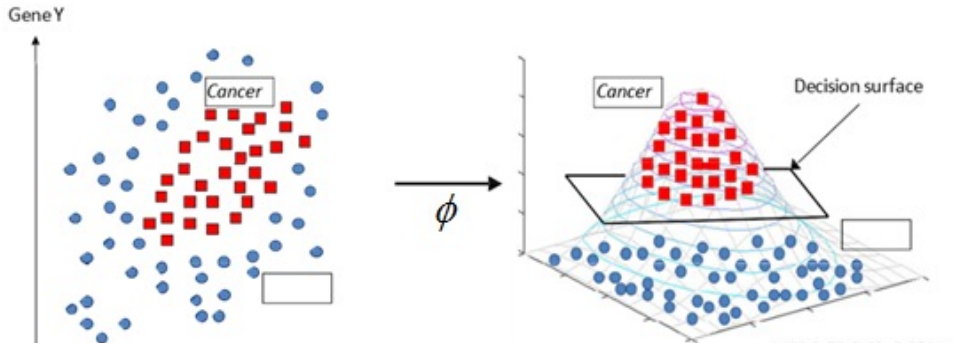
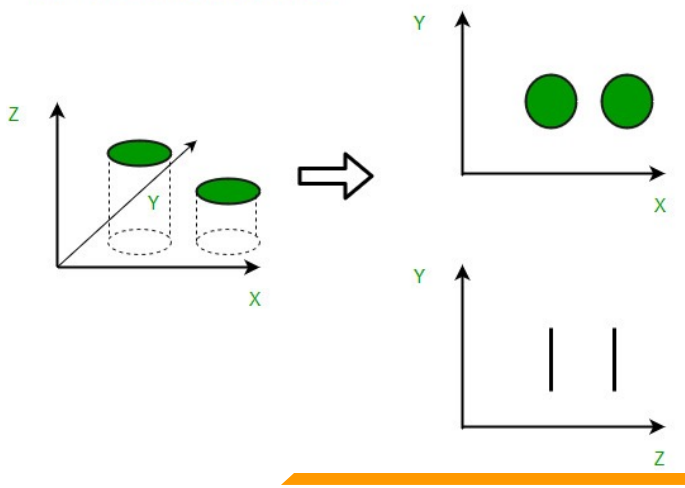
南方科技大学 SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY
地球与空间科学系 DEPARTMENT OF EARTH AND SPACE SCIENCES

Lesson 10: Dimension Reduction

CHEN Kejie 陈克杰（chenkj@sustech.edu.cn）

13-Apr-26





K-Nearest Neighbor, KNN

聚类的核心问题是什么？

不就是“谁跟谁像”吗？也就是说：相似的样本聚在一起。

在上一章我们讲了很多聚类方法（如**K-means**、**DBSCAN**等），你会发现一个共同点：

它们都依赖于“距离”来判断样本之间的相似性。

K-Nearest Neighbor, KNN

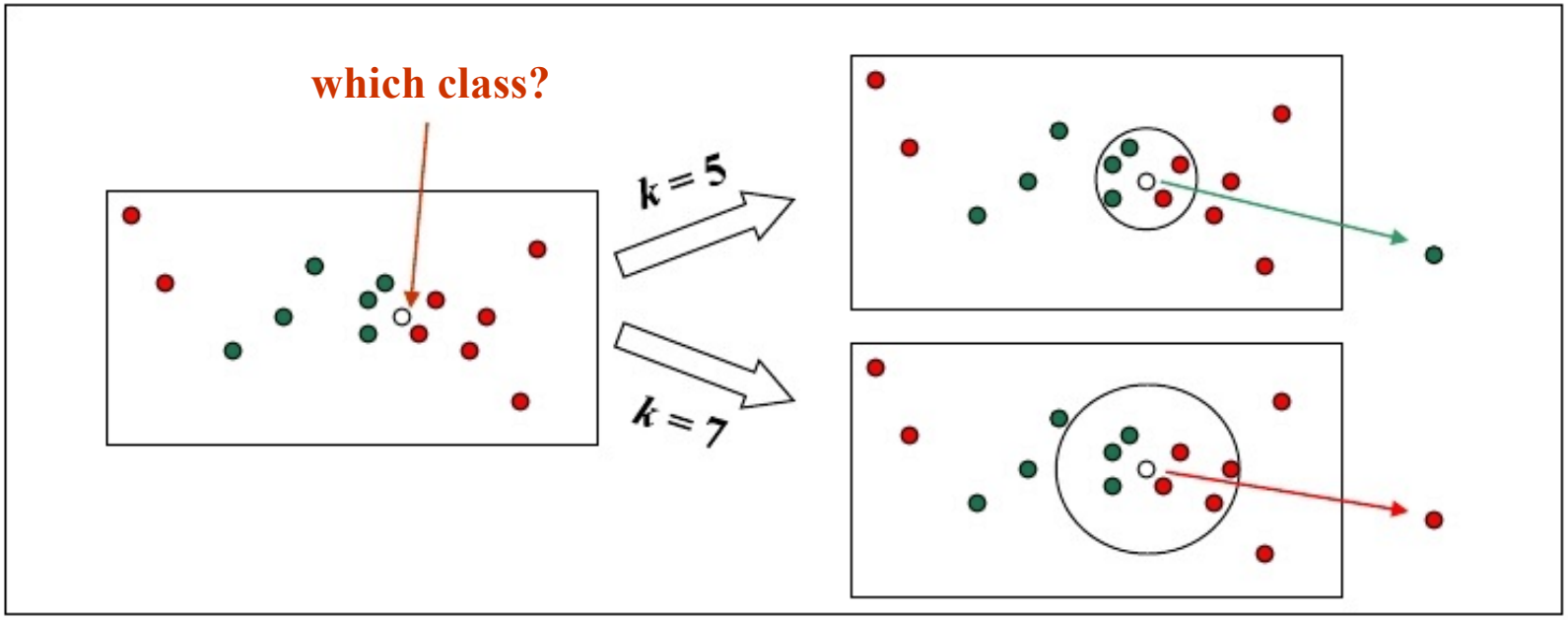
k 近邻 (k-Nearest Neighbor, kNN)

懒惰学习 (lazy learning) 的代表

最简单的有监督学习

基本思路：
近朱者赤，近墨者黑

(投票法；平均法)



关键： k 值选取；距离计算

K-Nearest Neighbor, KNN

给定测试样本 $\boldsymbol{x}$, 若其最近邻样本为 $\boldsymbol{z}$
$\boldsymbol{x}$和 $\boldsymbol{z}$类别标记不同的概率，

$$P(err) = 1 - \sum_{c \in \mathcal{Y}} P(c \mid \boldsymbol{x}) P(c \mid \boldsymbol{z}).$$

$$P(err) = 1 - \sum_{c \in \mathcal{Y}} P(c \mid \boldsymbol{x}) P(c \mid \boldsymbol{z})$$

最近邻分离器的泛化错误率不会超过贝叶斯最优分类器错误率的两倍

$$\begin{aligned} & 1 - \sum_{c \in \mathcal{Y}} P^2(c \mid \boldsymbol{x}) \\ \leq\ & 1 - P^2(c^* \mid \boldsymbol{x}) \\ =\ & \left(1 + P\left(c^* \mid \boldsymbol{x}\right)\right)\left(1 - P\left(c^* \mid \boldsymbol{x}\right)\right) \\ \leq\ & 2 \times \left(1 - P\left(c^* \mid \boldsymbol{x}\right)\right). \end{aligned}$$

但是在真实的应用中，我们是否能够准确的找到 k近邻呢？

Curse of Dimensionality 维数灾难

但是在真实的应用中，我们是否能够准确的找到 ***k***近邻呢？

密采样(dense sampling)

如果近邻的距离阈值设为 10^{-3}

假定维度为 20，如果样本需要满足密采样条件需要的样本数量近 10^{60}

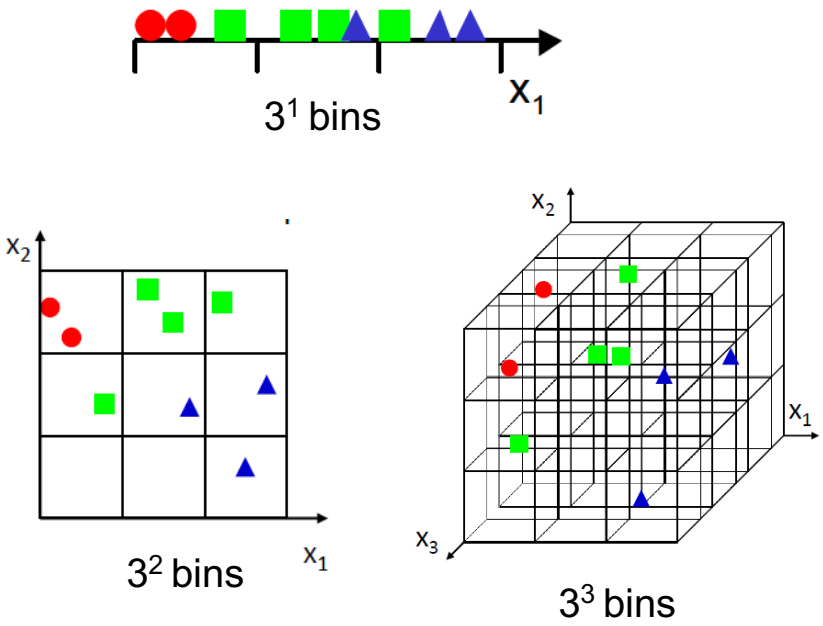
想象一下：一张并不是很清晰的图像： 70余万维

我们为了找到恰当的近邻，需要多少样本？

Curse of Dimensionality 维数灾难

- The number of training examples required increases exponentially with dimensionality **d** (i.e., k^d).

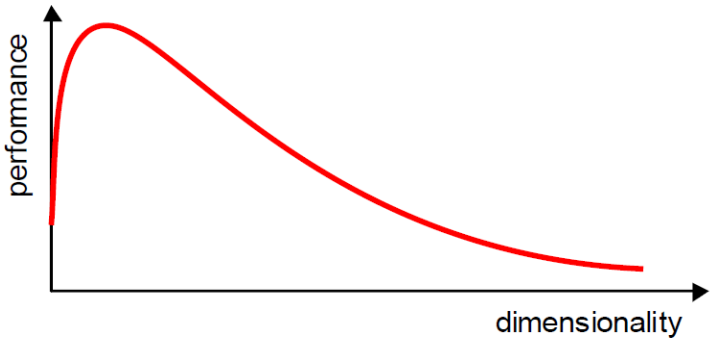
为了保持相同的“样本密度”，维度每升高一阶，样本量要指数级增加！



维度	每个维度分 3 个格子（bins）	总格子数
1维	$3^1 = 3$	3 个
2维	$3^2 = 9$	9 个
3维	$3^3 = 27$	27 个
d维	3^d	指数级增长！

Curse of Dimensionality 维数灾难

- Increasing the number of features will not always improve classification accuracy.



直觉上我们可能以为：“特征越多 → 信息越全 → 分类效果越好”。

但实际上，在高维空间中，增加特征往往不能提升模型性能，反而可能让性能变差，因为：

•多余的特征可能是噪声，影响模型的判断；

•高维下数据过于稀疏，难以学到可靠的统计规律。

Curse of Dimensionality 维数灾难

- In practice, the inclusion of more features might actually lead to worse performance.

不是所有特征都有助于分类！
尤其是在高维空间中：
•数据点之间距离都差不多；

- A = [1, 0, 0, ..., 0]（一个标准基）
- B = [1, 1, 1, ..., 1]（全1向量，维度为d）

我们计算两者的余弦相似度（Cosine similarity）：

$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{1}{\sqrt{1} \cdot \sqrt{d}} = \frac{1}{\sqrt{d}}$$

当维度 d 越大（比如128、512），这个值越小，表示它们变得“更不相关”

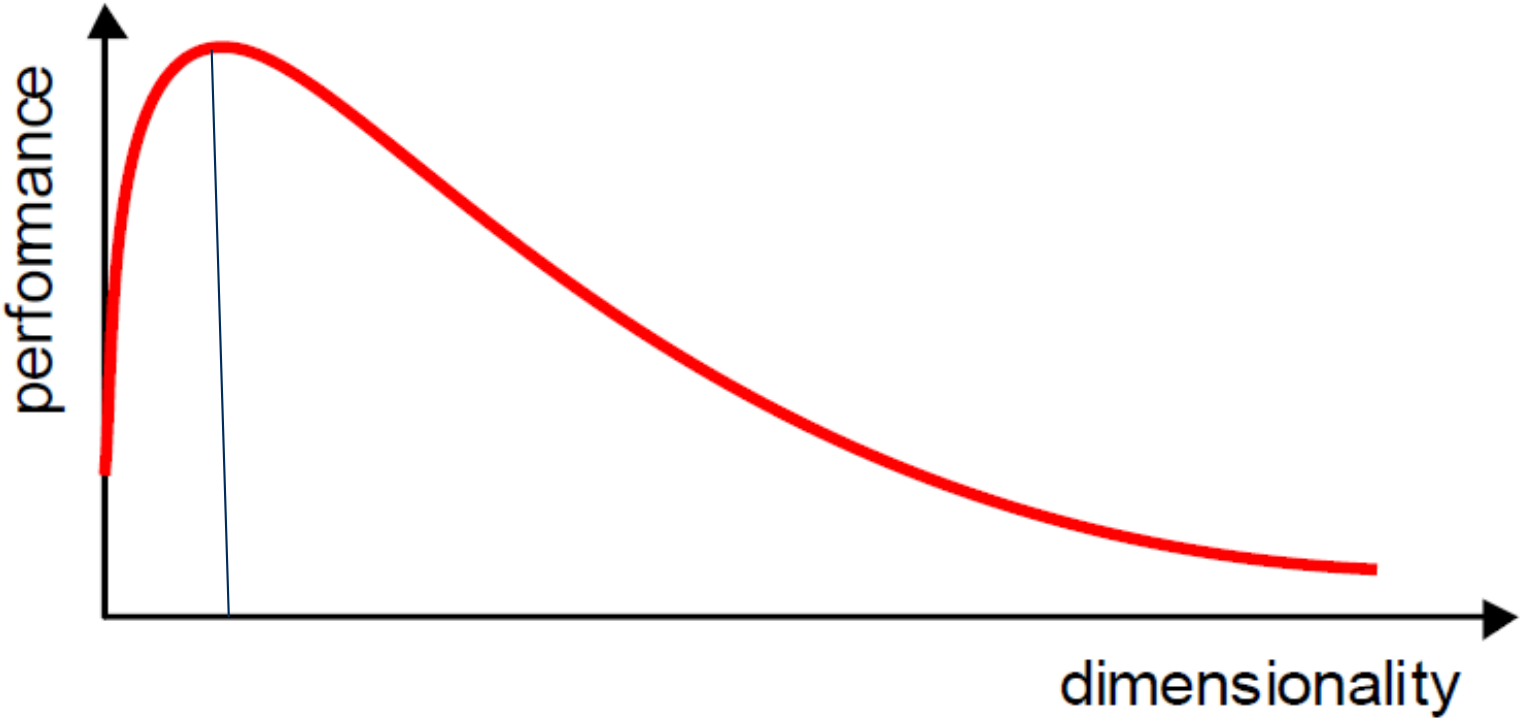
•导致模型很难判断“谁是近邻”；

•而KNN、决策树等方法严重依赖于“相似样本在一起”这个假设；

•结果是模型过拟合、泛化能力变差 → 性能变差。

Dimensionality Reduction

- What is the objective?
  - Choose an optimum set of features of lower dimensionality to improve classification accuracy.



Dimensionality Reduction (cont'd)

Feature selection（特征选择）: chooses a subset of the original features.

Feature extraction（特征抽取）: finds a set of new features (i.e., through some mapping f()) from the existing features.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdot \\ \cdot \\ \cdot \\ \cdot \\ x_N \end{bmatrix} \rightarrow \mathbf{y} = \begin{bmatrix} x_{i_1} \\ x_{i_2} \\ \cdot \\ \cdot \\ x_{i_K} \end{bmatrix}$$

K<<N

The mapping f() could be linear or non-linear

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdot \\ \cdot \\ \cdot \\ \cdot \\ x_N \end{bmatrix} \xrightarrow{f(\mathbf{x})} \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \cdot \\ \cdot \\ y_K \end{bmatrix}$$

K<<N

Feature Extraction

- Linear combinations are particularly attractive because they are simpler to compute and analytically tractable.

- Given $x \in R^N$, find an N x K matrix U such that:

$$y = U^T x \in R^K \text{ where } K<N$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ . \\ . \\ x_N \end{bmatrix} \xrightarrow{f(\mathbf{x})} \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_K \end{bmatrix}$$

U^T

This is a projection from the N-dimensional space to a K-dimensional space.

Feature Extraction (cont'd)

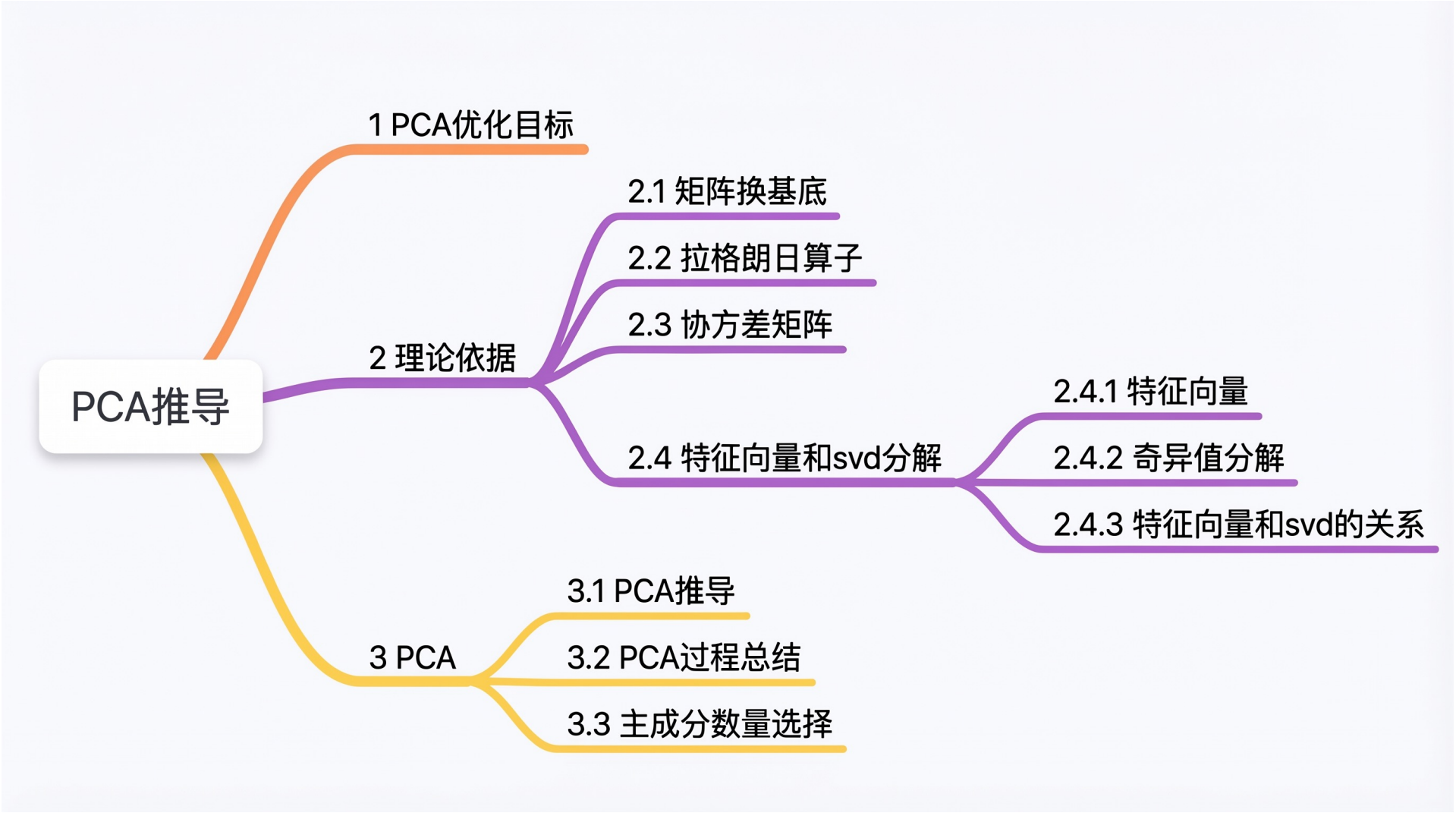
- From a mathematical point of view, finding an optimum mapping $\mathbf{y}=f(\mathbf{x})$ is equivalent to optimizing an **objective** function.

- Different methods use different objective functions, e.g.,
  - Information Loss: The goal is to represent the data as accurately as possible (i.e., no loss of information) in the lower-dimensional space.
  - Discriminatory Information: The goal is to enhance the class-discriminatory information in the lower-dimensional space.

Feature Extraction (cont'd)

- Commonly used linear feature extraction methods:
  - Linear Discriminant Analysis (LDA): Seeks a projection that **best discriminates** the data.
  - Principal Components Analysis (PCA): Seeks a projection that **preserves** as much **information** in the data as possible.
- Some other interesting methods:
  - Retaining interesting directions (Projection Pursuit),
  - Making features as independent as possible (Independent Component Analysis or ICA),
  - Embedding to lower dimensional manifolds (Isomap (等距特征映射) Locally Linear Embedding or LLE).

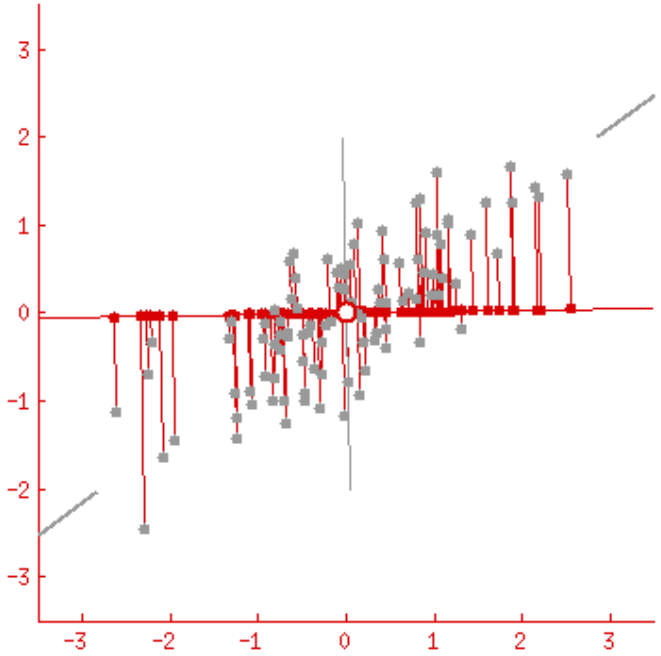
Principal Component Analysis (PCA)



PCA：优化目标

PCA推导有两种主要思路：
1.最大化数据投影后的的方差（让数据更分散）
2.最小化投影造成的损失

可证明两种思路是等价的，我们采用最大化方差推导：



投影到新坐标轴

投影损失

假设你把一个点 x 投影到了某个方向 u 上，投影点是 $\hat{x} = (u^\top x)u$。
那么损失就是 **原始点和投影点之间的欧几里得距离**：

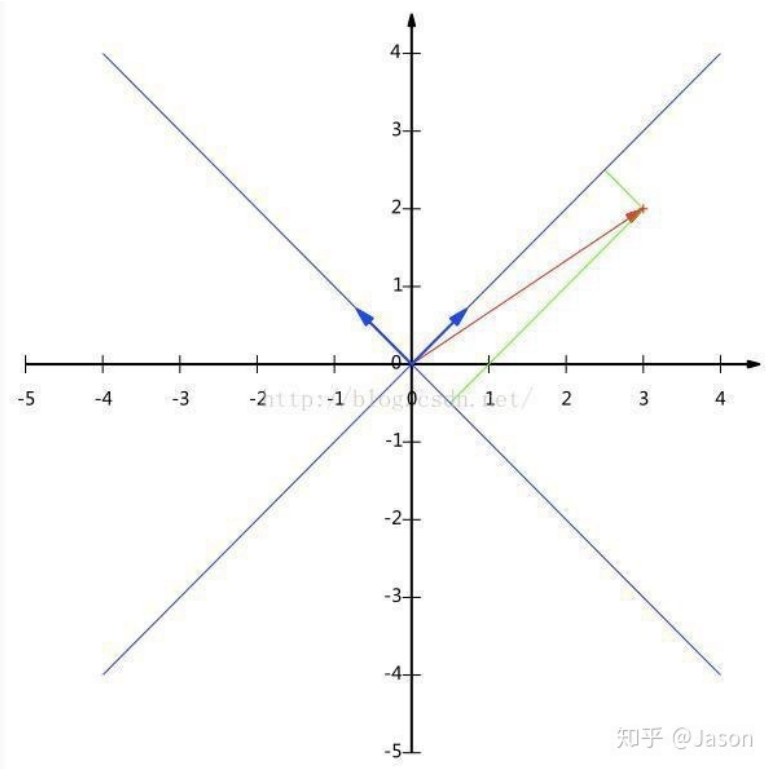
$$\text{Loss} = \|x - \hat{x}\|^2$$

对所有数据点求和后，得到整体的重建误差。

PCA：矩阵换基底

坐标变换地目标是，找到一组新的正交单位向量，替换原来的正交单位向量。下面通过具体例子说明。

假设存在向量 $\vec{a}=\begin{bmatrix}3\\2\end{bmatrix}$，要变换导以 $\vec{u}=\begin{bmatrix}\frac{1}{\sqrt{2}}\\\frac{1}{\sqrt{2}}\end{bmatrix}, \vec{v}=\begin{bmatrix}-\frac{1}{\sqrt{2}}\\\frac{1}{\sqrt{2}}\end{bmatrix}$ 为新基底地坐标上，求在心坐标系中的坐标



∵ 向量 $\vec{a}$ 在向量 $\vec{u}$ 上的投影距离 s：

$$s=||\vec{a}||\cdot\cos\theta=\frac{\vec{a}\cdot\vec{u}}{||\vec{u}||}=\vec{a}\cdot\vec{u}$$

其中：θ 表示两个向量之间的夹角

$$\therefore a_u=\vec{u}^T\cdot\vec{a},\ a_v=\vec{v}^T\cdot\vec{a}$$

∴ 向量 $\vec{a}$ 在新坐标系中的坐标可以表示为：

$$\vec{a}_{new}=[\vec{u}\quad\vec{v}]^T\cdot\vec{a}=\begin{bmatrix}\vec{u}^T\cdot\vec{a}\\\vec{v}^T\cdot\vec{a}\end{bmatrix}$$

如果矩阵 A 的列向量分别表示原来坐标系中的点，那么在新坐标系中的坐标为：

$$A_{new}=[\vec{u}\quad\vec{v}]^T\cdot A$$

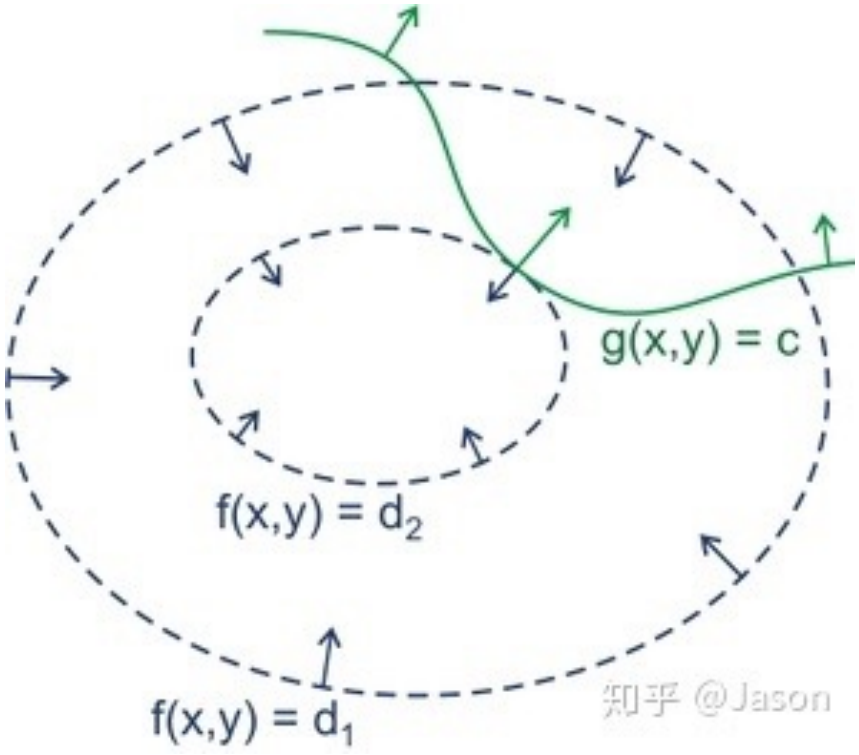
如果 $\vec{a}_{center}$ 表示一系列数据点的中心，那么可以证明：

$$\vec{a}_{newcenter}=[\vec{u}\quad\vec{v}]^T\cdot\vec{a}_{center}$$

PCA：拉格朗日乘子法

拉格朗日乘子法主要提供了一种求解函数在约束条件下极值的方法。下面还是通过一个例子说明。

假设存在一个函数 $f(x,y)$，求该函数在 $g(x,y)=c$ 下的极值（可以是极大，也可以极小）



通过观察我们发现，在极值点的时候两个函数必然相切，即此时各自的导数成正比，从而：

$$\frac{\partial f}{\partial x}=\lambda\frac{\partial g}{\partial x}$$
$$\frac{\partial f}{\partial y}=\lambda\frac{\partial g}{\partial y}$$
$$g(x,y)=c$$

通过联立上述三个公式，既可以求出最终结果。拉格朗日算子的主要思路同上，不过他假设了一个新的函数：

$$F(x,y,\lambda)=f(x,y)+\lambda[c-g(x,y)]$$

然后分解求：

$$\frac{\partial F}{\partial x}=0$$
$$\frac{\partial F}{\partial y}=0$$
$$\frac{\partial F}{\partial \lambda}=0$$

从而完成求解过程

PCA：协方差矩阵

假设有一组数据：

样本编号	变量x（如发传单数量）	变量y（如购买数量）	变量z（如购买总价）
1	1	2	3
2	35	25	55
...	...	...	...

协方差研究的目的是变量（特征）之间的关系，也就是上表中的发传单数量、购买数量、购买总额之间的相关情况。

上表数据用矩阵表示为：

$$X = \begin{bmatrix} 1 & 35 & \cdots \\ 2 & 25 & \cdots \\ 3 & 55 & \cdots \end{bmatrix}$$

那么两两变量之间的关系：

$$cov(x,y) = E[(1-E(x))(2-E(y)) + (35-E(x))(25-E(y)) + \cdots]$$
$$cov(x,z) = E[(1-E(x))(3-E(z)) + (35-E(x))(55-E(z)) + \cdots]$$
$$\cdots$$

如果E(x)=E(y)=E(z)=0（可以通过数据初始化实现），那么上述的协方差关系可以用如下矩阵乘法表示：

$$cov(X) = \frac{1}{m}XX^T = \begin{bmatrix} cov(x,x) & cov(x,y) & cov(x,z) \\ cov(y,x) & cov(y,y) & cov(y,z) \\ cov(z,x) & cov(z,y) & cov(z,z) \end{bmatrix}$$

PCA：奇异值分解

奇异值分解（svd: singular value decomposition）
定义：对于任意的矩阵A，存在：

$$A_{m\times n} = U_{m\times m} \cdot \Sigma_{m\times n} \cdot V^T_{n\times n}$$

其中：

$$U^T \cdot U = I_m$$
$$V^T \cdot V = I_n$$

即：U的列向量两两正交且模为1，V列向量两两正交且模为1，即：

$$U^T = U^{-1}$$
$$V^T = V^{-1}$$

对于任意矩阵 A，对A做svd有：

$$AA^T = U\Sigma V^T \cdot V\Sigma U^T = U\Sigma^2 U^{-1}$$

令 $\Sigma' = \Sigma^2$，则：

$$AA^T = U\Sigma' U^{-1}$$

满足$A = Q\Sigma Q^{-1}$特征向量定义

所以 AA^T 能实现特征分解，又因为：

$$AA^T = \underbrace{U''\Sigma''V''^T}_{svd}$$

奇异值分解肯定存在，
但不唯一

PCA：推导

PCA Objective: Maximizing Projected Variance

Find a new orthogonal basis $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_k\}$ (reducing from n to k dimensions) such that the projection of data points onto the subspace defined by this basis maximizes the distances (variance) between points.

Given an orthogonal basis vector $\boldsymbol{u}_j$, the projection of data point $\boldsymbol{x}_i$ is $\boldsymbol{x}_i^T \cdot \boldsymbol{u}_j$. The variance J_j of projections on this basis is:

$$J_j = \frac{1}{m} \sum_{i=1}^{m} (\boldsymbol{x}_i^T \cdot \boldsymbol{u}_j - \boldsymbol{x}_{\text{center}}^T \cdot \boldsymbol{u}_j)^2$$

Assumption: Data is centered ($\boldsymbol{x}_{\text{center}} = 0$):

$$J_j = \frac{1}{m} \sum_{i=1}^{m} (\boldsymbol{x}_i^T \cdot \boldsymbol{u}_j)^2 = \boldsymbol{u}_j^T \cdot \left[\frac{1}{m} \sum_{i=1}^{m} (\boldsymbol{x}_i \boldsymbol{x}_i^T) \right] \cdot \boldsymbol{u}_j$$

Matrix Notation:

$$J_j = \boldsymbol{u}_j^T \cdot \left[\frac{1}{m} \boldsymbol{X} \cdot \boldsymbol{X}^T \right] \cdot \boldsymbol{u}_j$$

Let $\boldsymbol{S} = \frac{1}{m} \boldsymbol{X} \cdot \boldsymbol{X}^T$ be the covariance matrix. Then:

$$J_j = \boldsymbol{u}_j^T \cdot \boldsymbol{S} \cdot \boldsymbol{u}_j$$

Optimization Goal:

To maximize the variance J_j, we need to find the vector $\boldsymbol{u}_j$ that maximizes $\boldsymbol{u}_j^T \cdot \boldsymbol{S} \cdot \boldsymbol{u}_j$.

PCA：推导

根据拉格朗日算子（求极值）求解：

$$J_j = u_j^T S u_j$$
$$s.t.\ u_j^T u_j = 1$$

则构造函数：

$$F(u_j) = u_j^T S u_j + \lambda_j (1 - u_j^T u_j)$$

求解 $\frac{\partial F}{\partial u_j} = 0$，得：

$$2S \cdot u_j - 2\lambda_j \cdot u_j = 0 \ \Rightarrow \ Su_j = \lambda_j u_j$$

结合2.4.1则：当 u_j、λ_j 分别为S矩阵的特征向量、特征值时， J_j 有极值，把上述结果带回公式得：

$$J_{j_m} = u_j^T \lambda_j u_j = \lambda_j$$

所以对于任意满足条件的正交基，对应的数据在上面投影后的方差值为S矩阵的特征向量，从而：

$$J_{max} = \sum_{j=1}^{k} \lambda_j$$ ，λ从大到小排序

PCA - Steps

Suppose we are given $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_M$ N x 1 vectors

Step 1: compute sample mean

$$\bar{\mathbf{x}} = \frac{1}{M}\sum_{i=1}^{M}\mathbf{x}_i$$

原始数据通常是一个「矩阵」：

设你有 M 个样本，每个样本有 N 个特征（像素、变量等），那原始数据形式是：

$$X = \begin{bmatrix} - & \mathbf{x}_1^T & - \\ - & \mathbf{x}_2^T & - \\ & \vdots & \\ - & \mathbf{x}_M^T & - \end{bmatrix} \in \mathbb{R}^{M\times N}$$

其中每一行是一个样本：$\mathbf{x}_i \in \mathbb{R}^N$，是**行向量**。

为了做PCA，我们通常会先把行向量转为列向量

PCA - Steps

Step 2: subtract sample mean (i.e., center data at zero)

$$\Phi_i = \mathbf{x}_i - \bar{\mathbf{x}}$$

减去均值也是减去列向量的均值

Step 3: compute the sample covariance matrix Σ_x

$$\Sigma_{\mathbf{x}} = \frac{1}{M}\sum_{i=1}^{M}\Phi_i\Phi_i^T$$

$$= \frac{1}{M}AA^T$$

where $A=[\Phi_1\ \Phi_2\ \ldots\ \Phi_M]$

(N x M matrix)

一共有 **M** 个样本（如**M**张图片）

PCA - Steps

Step 4: compute the eigenvalues/eigenvectors of Σ_x

$$\Sigma_x u_i = \lambda_i u_i$$

we assume $\lambda_1 > \lambda_2 > \ldots > \lambda_N \quad and \quad u_1, u_2, \ldots, u_N$ are the corresponding eigenvectors

Note : most software packages return the eigenvalues (and corresponding eigenvectors) is decreasing order – if not, you need to do it yourself)

Since Σ_x is symmetric, $\langle u_1, u_2, \ldots, u_N \rangle$ form an orthogonal basis in R^N, therefore:

$$\mathbf{x} - \bar{\mathbf{x}} = \sum_{i=1}^{N} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_N u_N$$

$$y_i = \frac{(\mathbf{x} - \bar{\mathbf{x}})^T u_i}{u_i^T u_i} = (\mathbf{x} - \bar{\mathbf{x}})^T u_i \qquad if \quad \| u_i \| = 1$$

Note : most software packages normalize u_i to unit length to simplify calculations; if not, you need to do it yourself):

PCA - Steps

Step 5: dimensionality reduction step – approximate **x** using only the first K eigenvectors (K<N) (i.e., corresponding to the K largest eigenvalues where K is a parameter):

$$\mathbf{x} - \bar{\mathbf{x}} = \sum_{i=1}^{N} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_N u_N$$

approximate using first K terms

$$\hat{\mathbf{x}} - \bar{\mathbf{x}} = \sum_{i=1}^{K} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_K u_K$$

or $$(\hat{\mathbf{x}} - \bar{\mathbf{x}}) = U \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_K \end{bmatrix}$$

$$where \;\; U = [u_1 \, u_2 \ldots u_K] \;\; N \, x \, K$$

i.e., the columns of U are the the first K eigenvectors of $\Sigma_{\mathbf{x}}$

PCA – Linear Transformation

- The linear transformation $R^N \rightarrow R^K$ which performs the dimensionality reduction is:

$\mathbf{y = U^T x} \in R^K$ where K<N

$$\begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_K \end{bmatrix} = U^T(\hat{\mathbf{x}} - \bar{\mathbf{x}}) = \begin{bmatrix} u_1^T \\ u_2^T \\ . \\ . \\ u_K^T \end{bmatrix} (\hat{\mathbf{x}} - \bar{\mathbf{x}})$$

i.e., the rows of U^T are the the first K eigenvectors of $\Sigma_{\mathbf{x}}$

Example

- Compute the PCA for dataset

(1,2),(3,3),(3,5),(5,4),(5,6),(6,5),(8,7),(9,8)

- Compute the sample covariance matrix is:

$$\hat{\Sigma} = \frac{1}{n}\sum_{k=1}^{n}(\mathbf{x}_k - \hat{\boldsymbol{\mu}})(\mathbf{x}_k - \hat{\boldsymbol{\mu}})^t$$

$$\Sigma_x = \begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}$$

- The eigenvalues can be computed by finding the roots of the characteristic polynomial:

$$\Sigma_x v = \lambda v \Rightarrow |\Sigma_x - \lambda I| = 0$$
$$\Rightarrow \begin{vmatrix} 6.25 - \lambda & 4.25 \\ 4.25 & 3.5 - \lambda \end{vmatrix} = 0$$
$$\Rightarrow \boldsymbol{\lambda_1 = 9.34; \lambda_2 = 0.41}$$

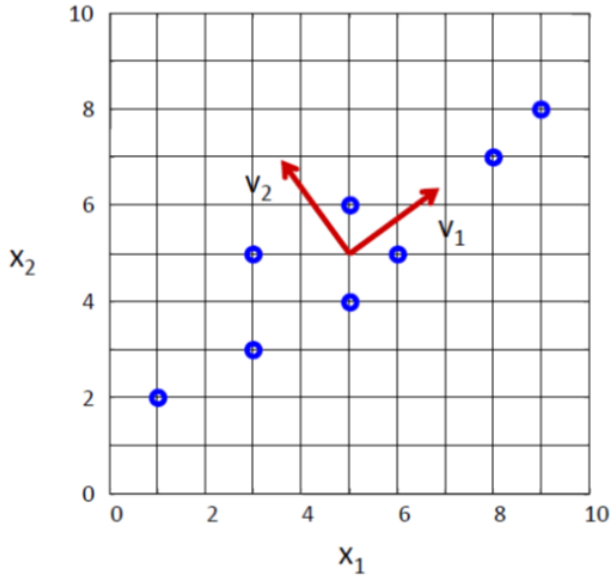
Example (cont'd)

- The eigenvectors are the solutions of the systems:

$$\Sigma_{\mathbf{x}} u_i = \lambda_i u_i$$

$$\begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} \lambda_1 v_{11} \\ \lambda_1 v_{12} \end{bmatrix} \Rightarrow \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0.81 \\ 0.59 \end{bmatrix}$$

$$\begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} \lambda_2 v_{21} \\ \lambda_2 v_{22} \end{bmatrix} \Rightarrow \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} -0.59 \\ 0.81 \end{bmatrix}$$



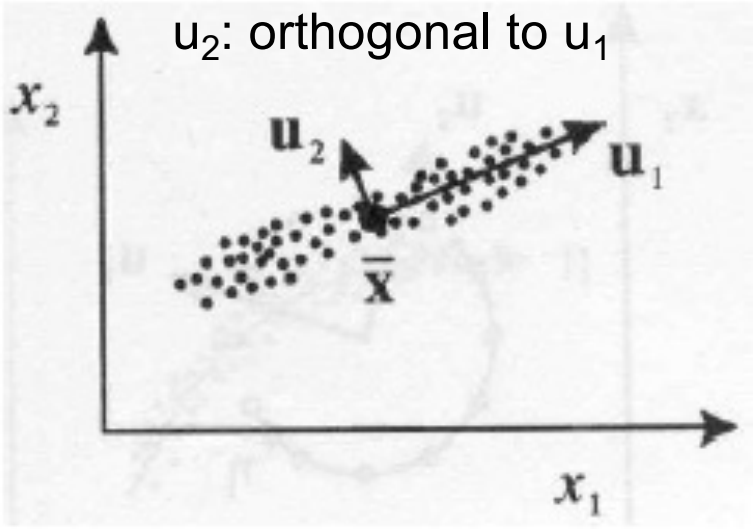
- Normalize the eigenvectors vectors to unit-length.

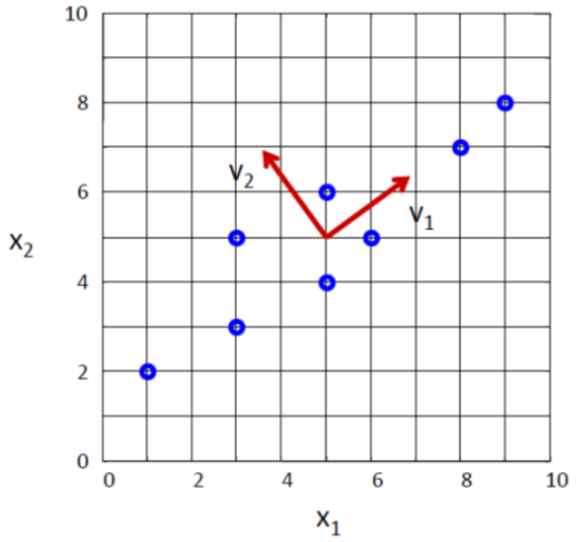
Note: if u_i is a solution, then (cu_i) is also a solution where c is any constant.

Geometric interpretation of PCA

- PCA chooses the eigenvectors of the covariance matrix corresponding to the largest eigenvalues.
- The eigenvalues correspond to the variance of the data along the eigenvector directions.
- Therefore, PCA projects the data along the directions where the data varies most.

u_1: direction of max variance
u_2: orthogonal to u_1





How do we choose K ?

- *K* is typically chosen based on how much information (variance) we want to preserve:

$$\frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{N} \lambda_i} > T \quad \textit{where } T \textit{ is a threshold } (e.g., 0.9)$$

- If T=0.9, for example, we say that we "preserve" 90% of the information (variance) in the data.

- If K=N, then we "preserve" 100% of the information in the data (i.e., just a change of basis)

Approximation Error

- The approximation error (or reconstruction error) can be computed as:

$$|| \mathbf{x} - \hat{\mathbf{x}} ||$$

where $\hat{\mathbf{x}} = \sum_{i=1}^{K} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_K u_K + \bar{\mathbf{x}}$ (reconstruction)

- It can also be shown that the approximation error can be computed as follows:

$$|| \mathbf{x} - \hat{\mathbf{x}} || = \frac{1}{2} \sum_{i=K+1}^{N} \lambda_i$$

Data Normalization

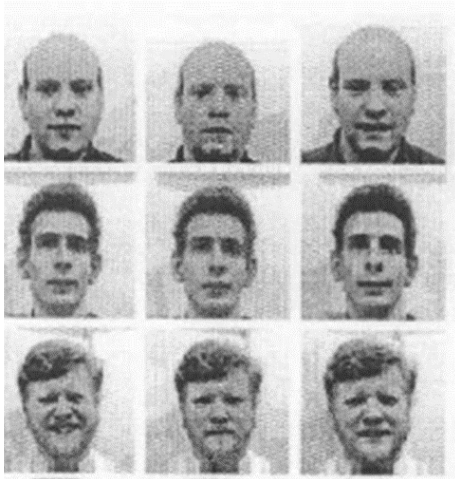
- The principal components are dependent on the ***units*** used to measure the original variables as well as on the ***range*** of values they assume.
- Data should always be normalized prior to using PCA.
- A common normalization method is to transform all the data to have zero mean and unit standard deviation:

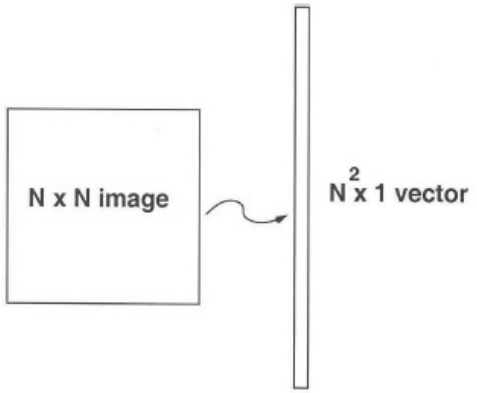
$$\frac{x_i - \mu}{\sigma}$$

where μ and σ are the mean and standard deviation of the i-th feature x_i

Application to Images – Practical Issues

- The goal is to represent images in a space of lower dimensionality using PCA.
  - Useful for various applications, e.g., face recognition, image compression, etc.
- Given M images of size N x N, first represent each image as a 1D vector (i.e., by stacking the rows together).
  - Note that for face recognition, faces must be centered and of the same size.





Application to Images (cont'd)

- The key challenge is that the covariance matrix Σ_x is now very large (i.e., N^2 x N^2) – see Step 3:

Step 3: compute the covariance matrix Σ_x

$$\Sigma_{\mathbf{x}} = \frac{1}{M}\sum_{i=1}^{M}\Phi_{\mathbf{i}}\Phi_i^T = \frac{1}{M}AA^T$$

where $A=[\Phi_1\ \Phi_2\ \ldots\ \Phi_M]$ (N^2 x M matrix)

- $\Sigma_{\mathbf{x}}$ is now an N^2 x N^2 matrix – computationally expensive to compute its eigenvalues/eigenvectors λ_i, u_i

$$(AA^T)u_i = \lambda_i u_i$$

Application to Images (cont'd)

- We will use a simple "trick" to get around this!

- Let us consider the matrix A^TA instead (i.e., M x M matrix)
  - Suppose its eigenvalues/eigenvectors are μ_i, v_i

$$(A^TA)v_i = \mu_i v_i$$

  - Multiply both sides by A:

$$A(A^TA)v_i = A\mu_i v_i \quad \text{or} \quad \boxed{(AA^T)(Av_i) = \mu_i(Av_i)}$$

  - Since $(AA^T)u_i = \lambda_i u_i$

$$\boxed{\lambda_i = \mu_i \quad \text{and} \quad u_i = Av_i}$$

$A=[\Phi_1\ \Phi_2\ \ldots\ \Phi_M]$
(N^2 x M matrix)

Application to Images (cont'd)

- But do AA^T and A^TA have the same number of eigenvalues/eigenvectors?
  - AA^T can have up to N^2 eigenvalues/eigenvectors.
  - A^TA can have up to M eigenvalues/eigenvectors.
  - It can be shown that the M eigenvalues/eigenvectors of A^TA are also the M largest eigenvalues/eigenvectors of AA^T
- **Steps 3-5** of PCA need to be updated as follows:

Application to Images (cont'd)

Step 3 compute A^TA (i.e., instead of AA^T)

Step 4: compute μ_i, v_i of A^TA

Step 4b: compute λ_i, u_i of AA^T using $\lambda_i=\mu_i$ and $u_i=Av_i$, then normalize u_i to unit length.

Step 5: dimensionality reduction step – approximate **x** using only the first K eigenvectors (K<M):

$$\hat{\mathbf{x}} - \overline{\mathbf{x}} = \sum_{i=1}^{K} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_K u_K$$

each image can be represented by a K-dimensional vector

$$\Omega = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_K \end{bmatrix}$$

Example

Dataset

Example (cont'd)

Top eigenvectors: $\mathbf{u}_1,\ldots\mathbf{u}_k$

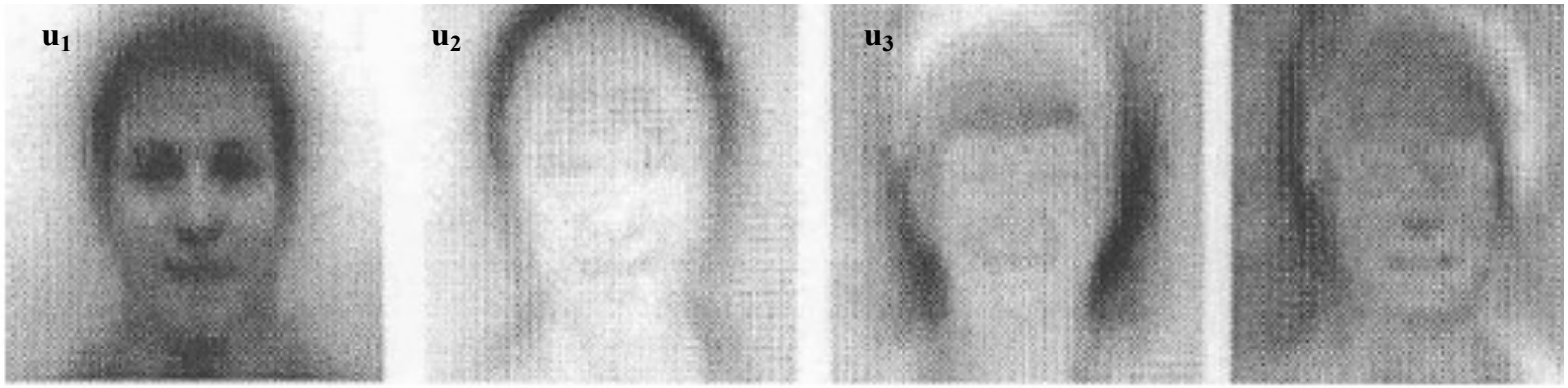
(visualized as images - eigenfaces)

Mean face: $\bar{\mathbf{x}}$

主成分（PCA）	捕捉的内容（可能）	看起来是否"有脸"
u_1、u_2	背景亮度、全局灰度偏移、脸的模糊形状	否 / 模糊
u_{10} 等	局部面部结构：眼睛、鼻子、嘴等	是 / 清晰

Application to Images (cont'd)

- Interpretation: represent a face in terms of eigenfaces



$$\hat{\mathbf{x}} = \sum_{i=1}^{K} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_K u_K + \bar{\mathbf{x}}$$

$$\Omega = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_K \end{bmatrix}$$

Case Study: Dog Image Compression